2-3

Streamlit을 활용한 RAG 배포

학습 목차

- **Streamlit**
  - Streamlit 소개
  - Python 코드 기반 웹 UI
- **Streamlit 앱 기본 구성**
  - title과 caption
  - cache_resource
- **채팅 인터페이스 구성**
  - session_state
  - chat_message와 chat_input
  - write_stream

© Scopelabs. All Rights Reserved

Streamlit

Streamlit 소개

Streamlit: Python 코드만으로 빠르게 웹 애플리케이션을 만들 수 있게 해주는 프레임워크

- HTML, CSS, JavaScript를 직접 작성하지 않아도 됨
- Python 함수와 변수를 화면 요소로 바로 연결 가능
- 데이터 분석, AI 데모, 내부 업무 도구를 빠르게 만들 때 적합

LLM 실습에서 Streamlit을 사용하는 이유

- 콘솔 실행 결과를 사용자가 보기 쉬운 화면으로 바꿀 수 있음
- 질문 입력창과 답변 영역을 간단히 만들 수 있음
- 모델 응답이 생성되는 과정을 스트리밍으로 보여줄 수 있음
- 실습한 RAG Chain을 웹 UI와 연결해 서비스 형태로 확인할 수 있음

Python 코드 기반 웹 UI

Streamlit 앱은 위에서 아래로 실행되는 Python 스크립트처럼 작성

- `streamlit run app.py`로 실행
- 코드가 실행되며 화면 요소가 순서대로 그려짐
- 사용자 입력이 발생하면 스크립트가 다시 실행되는 구조

일반 웹 앱과 다르게 생각해야 할 점

- 버튼 클릭이나 입력이 있을 때 전체 스크립트가 다시 실행될 수 있음
- 유지해야 하는 값은 `st.session_state`에 저장
- 오래 걸리는 초기화 작업은 cache 기능으로 재사용

Streamlit 앱 기본 구성

title과 caption (1)

`st.title`: 앱의 가장 큰 제목을 화면에 표시

- 사용자가 어떤 앱을 보고 있는지 바로 이해하게 함
- 실습 코드에서는 `보험 약관 QA 챗봇`을 제목으로 표시
- 화면의 첫 인상을 결정하는 가장 기본적인 UI 요소

```
import streamlit as st

st.title("보험 약관 QA 챗봇")
```



title과 caption (2)

`st.caption`: 제목 아래에 짧은 보조 설명을 표시

- 앱이 어떤 데이터를 기반으로 답변하는지 안내
- 실습 코드에서는 `메리츠 펫보험 약관을 기반으로 답변합니다.`를 표시
- 제목보다 작고 가벼운 설명에 적합

```
st.caption("메리츠 펫보험 약관을 기반으로 답변합니다.")
```



cache_resource

`@st.cache_resource`: 무거운 리소스를 한 번만 만들고 재사용하기 위한 데코레이터

- 모델, 벡터스토어, DB 연결처럼 초기화 비용이 큰 객체에 사용
- Streamlit이 재실행되어도 같은 리소스를 다시 만들지 않도록 캐싱
- 실습 코드에서는 `load_rag_chain()`에 적용

```
@st.cache_resource
def load_rag_chain():
    loader = PDFMinerLoader("./data/meritz_pet_insurance.pdf")
    documents = loader.load()
    vectorstore = Chroma.from_documents(documents, embeddings)
    return create_rag_chain(vectorstore.as_retriever())
```

채팅 인터페이스 구성

session_state

`st.session_state`: 사용자 세션 동안 유지해야 하는 값을 저장하는 공간

- Streamlit 앱은 입력이 발생할 때 다시 실행될 수 있음
- 일반 변수만 사용하면 이전 대화가 사라질 수 있음
- 대화 기록처럼 유지해야 하는 값은 `session_state`에 저장
  - Streamlit의 특성상, `session_state`에 저장된 메시지를 매번 다시 출력해야 함

```
if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])
```

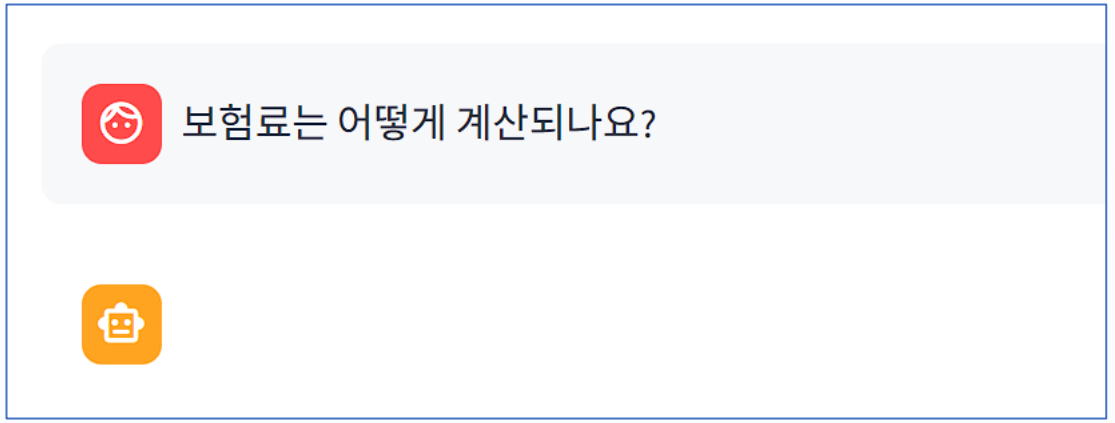
`chat_message`와 `chat_input`

`st.chat_message`: 채팅 메시지를 역할별 말풍선 형태로 표시

- `user` 역할은 사용자 메시지 영역
- `assistant` 역할은 AI 답변 영역

`st.chat_input`: 사용자 질문을 입력받는 채팅 입력창

- 화면 하단에 입력창을 제공
- 사용자가 입력하고 Enter를 누르면 문자열을 반환




```
if prompt := st.chat_input("질문을 입력하세요"):
    st.session_state.messages.append({
        "role": "user",
        "content": prompt,
    })

    with st.chat_message("user"):
        st.write(prompt)
```

write_stream

`st.write_stream`: 생성되는 텍스트를 한 번에 보여주지 않고 순차적으로 화면에 출력

- LLM이 답변을 생성하는 과정을 실시간으로 보여줄 수 있음
- 사용자는 응답이 시작됐다는 느낌을 바로 받을 수 있음
- 긴 답변일수록 체감 대기 시간이 줄어듦

```
with st.chat_message("assistant"):
    rag_chain = load_rag_chain()
    response = st.write_stream(
        rag_chain.stream(prompt)
    )

st.session_state.messages.append({
    "role": "assistant",
    "content": response,
})
```